

Architecture Design Specification: Hybrid Instruction Set Computing (HISC) System

1. Architectural Vision and Strategic Rationale

The Hybrid Instruction Set Computing (HISC) system is a strategic architectural mandate necessitated by the plateauing of Moore's Law and the collapse of Dennard scaling. For decades, the industry was bifurcated by the "ISA wars," which created a false dichotomy between the streamlined execution of Reduced Instruction Set Computing (RISC) and the rich code density of Complex Instruction Set Computing (CISC). Modern high-performance and edge computing environments can no longer afford this binary approach. HISC synthesizes these philosophies, recognizing that the fundamental tension—RISC's pipelining efficiency versus CISC's memory-efficient variable-length instructions—is a microarchitectural optimization problem rather than a theological divide. **The "So What?"**

Factors: Strategic Value Proposition

- **Silicon Utilization and PPA:** HISC mandates the use of asymmetric core clusters to maximize Power-Performance-Area (PPA). By reclaiming die area from underutilized logic, we significantly increase core density.
- **Energy Constraints in IoT:** The architecture prioritizes energy efficiency for edge devices, leveraging Partial-ISA cores to reduce the power footprint while maintaining the capability to handle complex bursts on demand.
- **AI and HDC Intensity:** Emerging paradigms like Hyperdimensional Computing (HDC) and Convolutional Neural Networks (CNN) require specialized arithmetic that standard rigid ISAs cannot provide. HISC integrates domain-specific custom instructions to bridge this gap. Achieving this vision requires a fundamental transition from homogeneous core clusters to a specialized, asymmetric RISC-CISC matrix.

2. Core Cluster Heterogeneity: The RISC-CISC Matrix

The HISC architecture maximizes system throughput by employing asymmetric functionality via Partially Heterogeneous ISA (PHISA) clusters. This is fundamentally an **Area (A) saving strategy**. By removing costly, specialized instruction extensions from a portion of the cores, we reclaim up to **69% of the core area** (typical of out-of-order NEON units). This allows the design to triple or quadruple the count of in-order efficiency cores within the same reticle limit, significantly increasing Task Parallelism (TP). The following matrix compares the core types within the cluster, distinguishing between the Base ISA (the software contract) and the internal Micro-ops (the execution substrate).

Feature	High-Density CISC Cores	Streamlined RISC Cores
Philosophical Role	Legacy density and complex binary compatibility management.	High-speed, primary execution substrate for parallel tasks.
Instruction Handling	Translates variable-length CISC instructions into RISC-like micro-ops.	Native, single-cycle execution of orthogonal instructions.
Microarchitectural Lineage	NexGen/P6-style "CISC-on-RISC" translation front-end.	RISC-V / ARM (Cortex-A/M) performance-per-watt leaders.
Pipeline Target	Rich decode logic with high-capacity micro-op translation.	Deep pipelining with minimalist, hardwired control logic.
Silicon Overhead	High; utilizes full ISA extensions (e.g., NEON/AVX).	Low; implements "Partial-ISA" to maximize core density.

The physical diversity of these cores necessitates a sophisticated Hardware Abstraction Layer to maintain software transparency while managing hardware asymmetry.

3. The Hardware Abstraction Layer (HAL) Design

The Hardware Abstraction Layer (HAL) functions as the contractual gatekeeper between software intent and hardware execution. It ensures that the software remains agnostic of the underlying silicon topology, even when the ratio of RISC-to-CISC cores varies between product SKUs. The HAL architecture is defined by three pillars:

1. **Hardware Independence:** Standardized APIs hide whether an operation is dispatched to a RISC or CISC resource.
2. **Portability:** Software remains binary-compatible across the HISC roadmap; only the HAL implementation is adapted to new silicon configurations.
3. **Consistency:** Predictable performance is maintained by masking the hardware's heterogeneous routing logic. **Interoperability and Memory Coherency** The HAL manages a Unified Memory Architecture where all cores share a single physical DRAM pool. This Interoperability Layer is responsible for strict data consistency and coherency management during thread migration, ensuring that memory state is preserved when a task moves between different ISA core types.

4. Instruction Pipeline Optimization: Fusion and Micro-Ops

To bridge the performance gap between CISC density and RISC speed, HISC employs aggressive internal instruction transformations. These mechanisms ensure that simple instructions achieve the throughput of complex machines.

Macro-op Fusion

Macro-op fusion identifies adjacent RISC-V instructions (e.g., an ADD followed by a LOAD) and coalesces them in the decode stage into a single "macro-op." This fused operation is dispatched and retired atomically, easing pressure on the reorder buffer and backend units. Benchmarks demonstrate that fusion can achieve up to a **19.6% reduction** in dynamic operation counts for workloads like bzip2.

Micro-op Cache and Compaction

For CISC-to-RISC translation, the system utilizes a Micro-op (uop) cache to bypass energy-intensive decoders. To maximize hit rates and minimize fragmentation, the following optimizations are mandated:

- **CLASP (Cache Line boundary AgnoStic uoP):** This design allows uop sequences to span across l-cache line boundaries, preventing the arbitrary termination of sequential code.
- **Compaction Strategies:**
- **RAC (Replacement policy Aware Compaction):** Compacts new entries with Most Recently Used (MRU) entries to ensure temporal correlation.
- **PWAC (Prediction Window Aware Compaction):** Compacts entries belonging to the same branch prediction window.
- **F-PWAC (Forced PW-Aware Compaction):** Forces compaction of entries from the same window into a single line to maximize fetch bandwidth. These pipeline optimizations reduce decoder power consumption by up to **31.53%** and significantly lower branch misprediction latency by shortening the pipeline depth.

5. Workload Orchestration and Intelligent Scheduling

The scheduler is the operational brain of the HISC system, balancing the conflicting requirements of peak single-thread performance and cluster-wide energy efficiency.

The Fault-and-Migrate Protocol

When a Partial-ISA core encounters an unsupported opcode (e.g., a SIMD instruction), the system triggers the following protocol:

1. **Trap:** The core generates an unsupported-instruction exception.
2. **OS Signal:** The OS intercepts the trap (e.g., TRAP UD).
3. **Migration:** The scheduler moves the thread to a Full-ISA core.

Migration Dynamics and Residency

The system must minimize "oscillation," where threads bounce between cores. The HISC scheduler mandates a **160K cycle minimum residency threshold**; a thread migrated to a Full-ISA core must remain there for at least 160,000 cycles before being considered for a move back to an efficiency core. This amortizes the overhead of context switching. While software emulation is possible, it is strategically avoided for intensive tasks, as emulation typically costs **40x more cycles** than hardware execution.

Policy Framework

- **Performance Optimization Policy:** Prioritizes Out-of-Order (OoO) Full-ISA cores for maximum single-thread speed.
- **Energy Efficiency Policy:** Prioritizes In-order Partial-ISA cores, leveraging research showing an **82% energy reduction** when offloading basic tasks from power-hungry cores.

6. Domain-Specific Acceleration: HDC-CNN Hybridization

HISC provides the foundation for custom accelerators targeting Hyperdimensional Computing (HDC). By implementing four specialized instructions—Binding, Bundling, Similarity, and Binarize—the RISC-V GPU components achieve a **56.2x speedup** in microbenchmarks.

The Encoding Bottleneck

While HDC operations are accelerated, real-world image classification is limited by the **Encoding Bottleneck**. This bottleneck is caused by **Locality-based Sparse Random Projection**, which is heavily dependent on matrix operations. Current benchmarks show that this overhead limits real-world speedup to approximately 2% if not properly addressed.

Evolutionary Path: Integrated Matrix Extensions

To resolve the encoding bottleneck, the HISC architecture mandates the integration of **Integrated Matrix Extensions**. By combining these extensions with specialized HDC logic, the system overcomes the matrix operation overhead, fully unlocking the potential of brain-inspired AI models. The HISC architecture represents the definitive evolutionary step for post-Moore silicon, providing a scalable, energy-efficient platform that reclaims die area while delivering specialized performance for the next generation of AI and edge computing.